

OUTLINE

An overview of today's presentation plan:

- Project Introduction
- Theoretical Background
- Simulink Simulation
- Design and Microcontroller Programming
- Control Interface Programming
- Results and Experimentation



Project Introduction

Design of a PI-MRAS Controller for DC Motor Control

OBJECTIVES

- Design a speed controller that enables the motor speed to track a given reference model.
- Ensure system performance in terms of overshoot, settling time, and stability.
- Develop a user interface for monitoring and control via a PC.



Theoretical Background

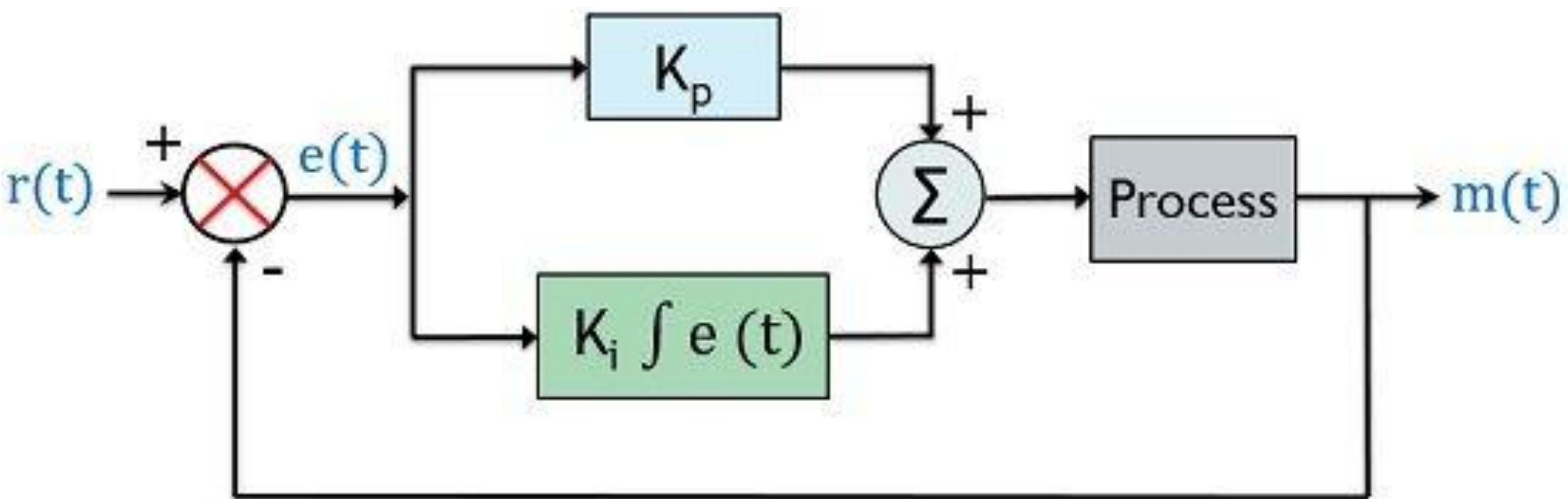
01

DC MOTOR SPEED CONTROL – PI CONTROLLER

02

DC MOTOR SPEED CONTROL – PI MRAS
CONTROLLER





Control System with PI Controller

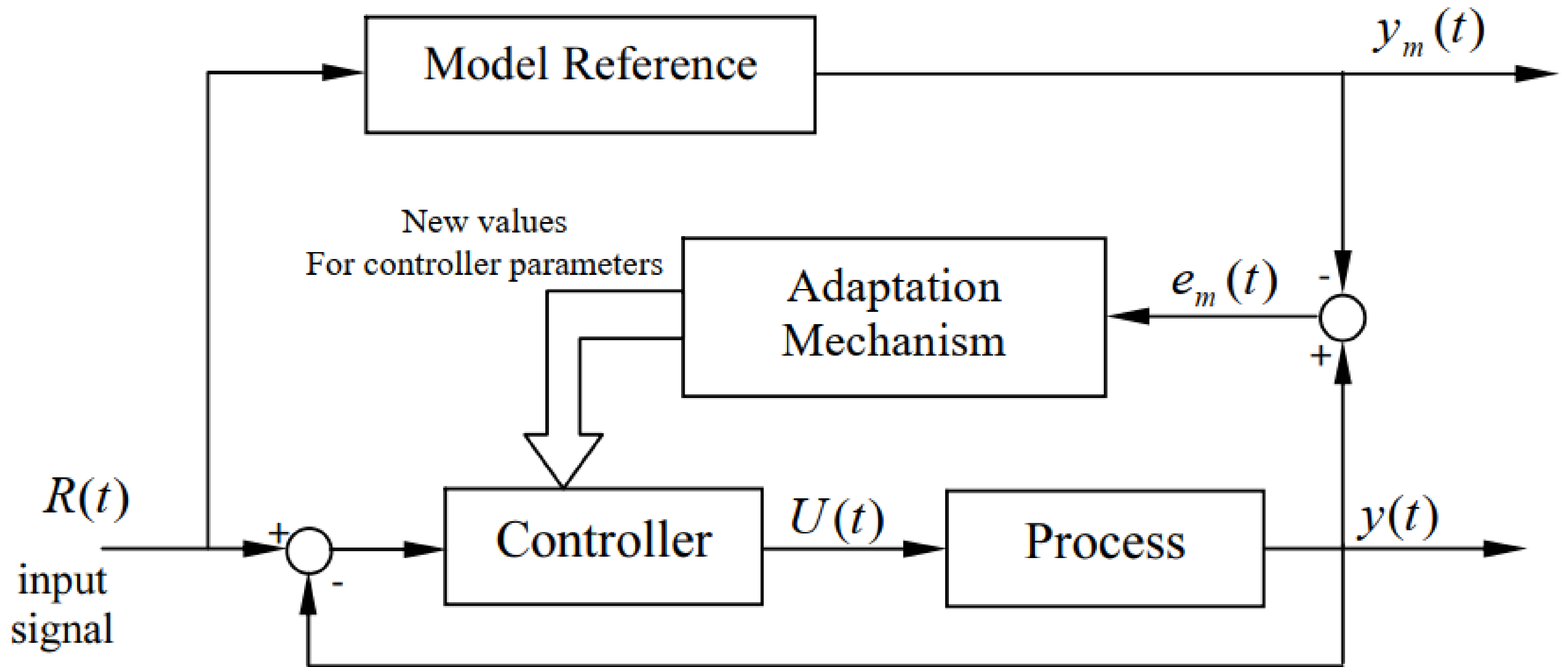
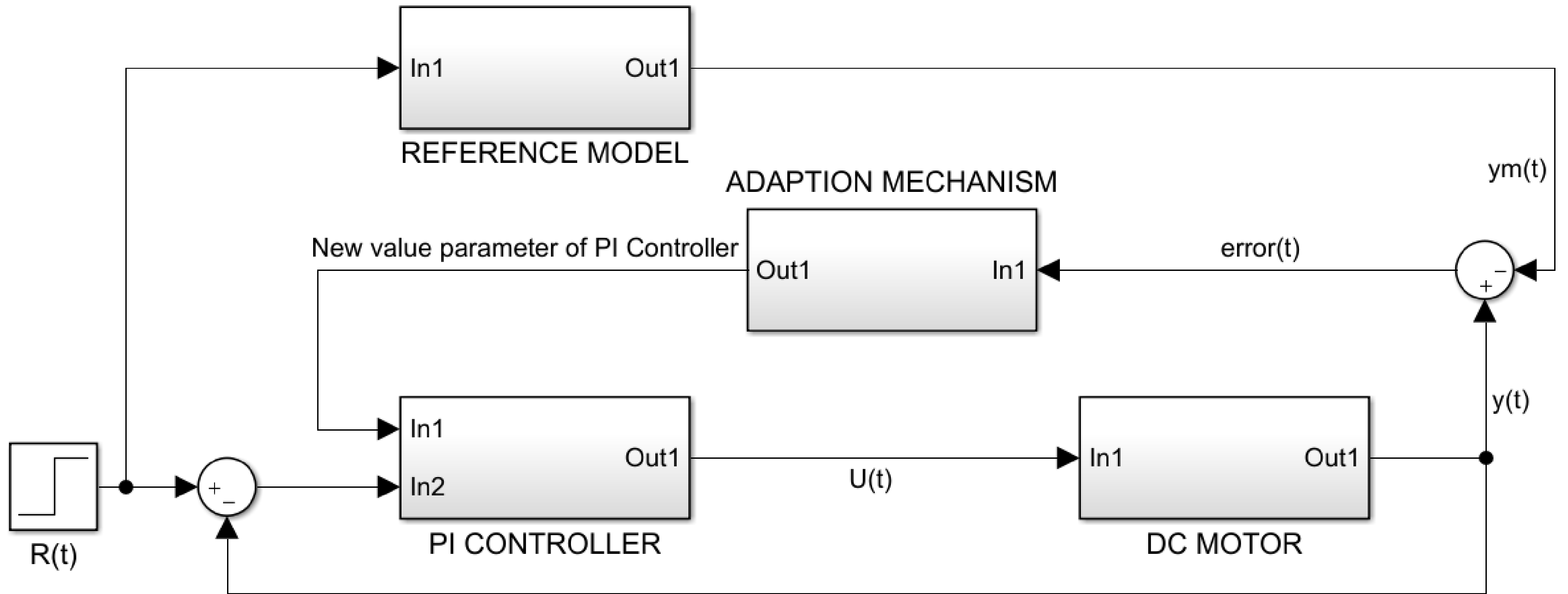


Fig. 5 Block diagram of MRAC.



$$J = \frac{1}{2} \int_0^t e_m^2(\tau) d\tau$$

Where $e_m(t) = y(t) - y_m(t)$

The adjustment of the controller parameters is made according to the following equation, which is called MIT rule [1]:

$$\frac{d\theta_i}{dt} = -\gamma_i \frac{d}{dt} \left(\frac{\partial J}{\partial \theta_i} \right) \quad (1)$$

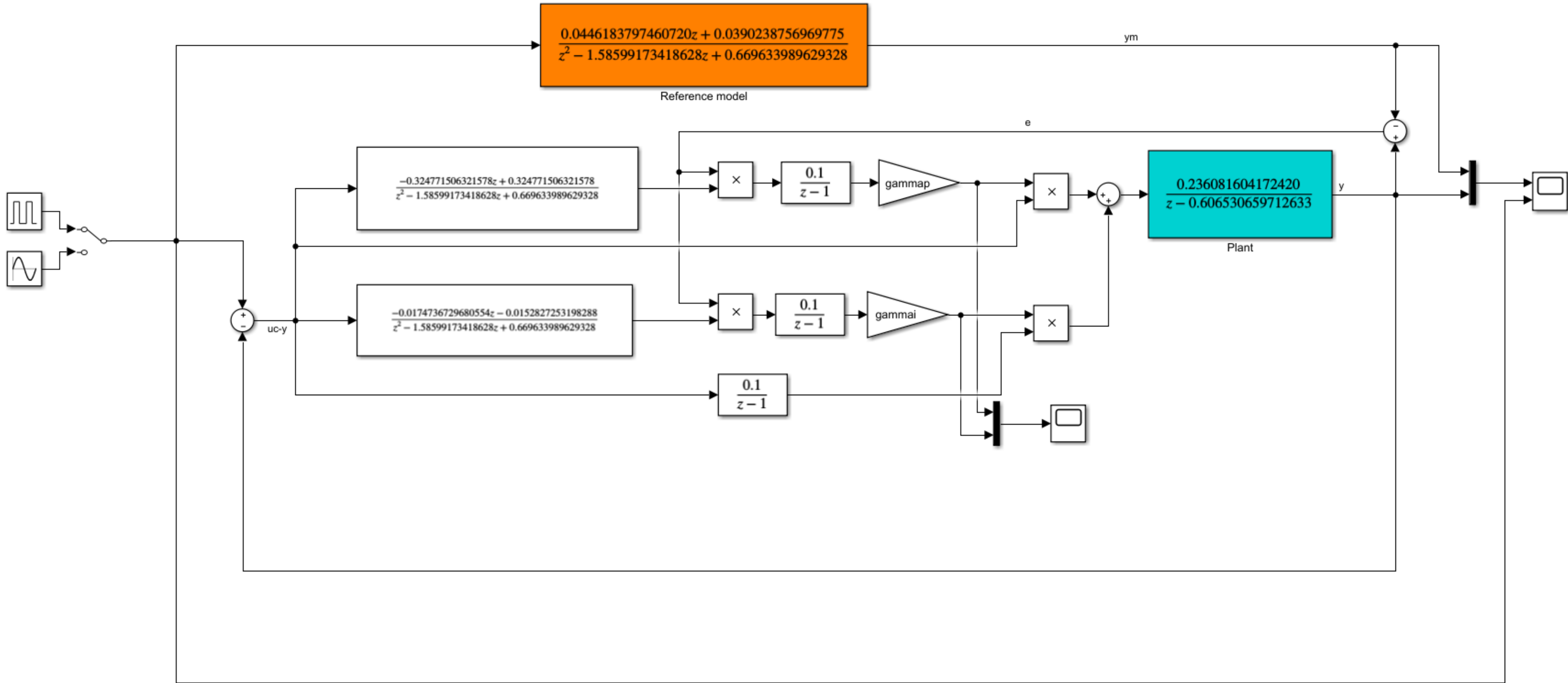
Equation (1) represents the general adaptive law using gradient method.

$$\dot{k}_p(t) = -\gamma'_1 e_m(t) \frac{\partial y}{\partial k_p} \longrightarrow k_p(s) = -\gamma_1 \cdot e_m(s) \cdot G_m(s) \cdot [R(s) - Y(s)]$$

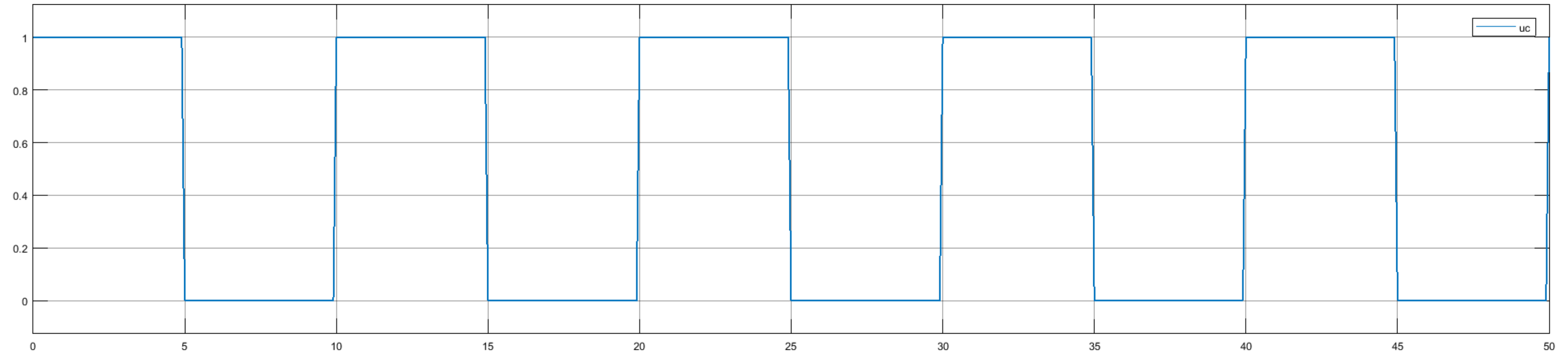
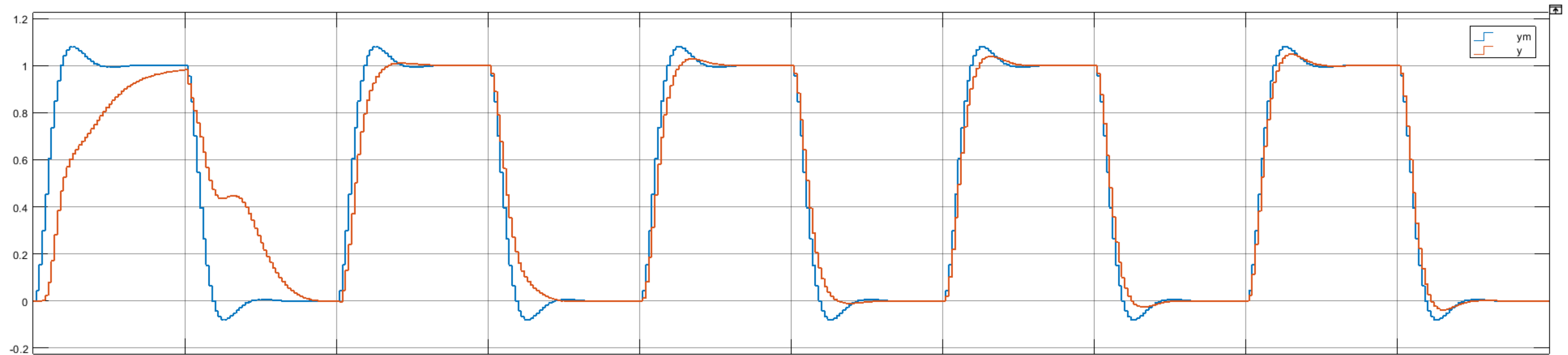
$$\dot{k}_I(t) = -\gamma'_2 e_m(t) \frac{\partial y}{\partial k_I} \longrightarrow k_I(s) = -\frac{\gamma_2 \cdot e_m(s) \cdot G_m(s)}{s} \cdot [R(s) - y(s)]$$

Where y(t) is a function of output DC Motor

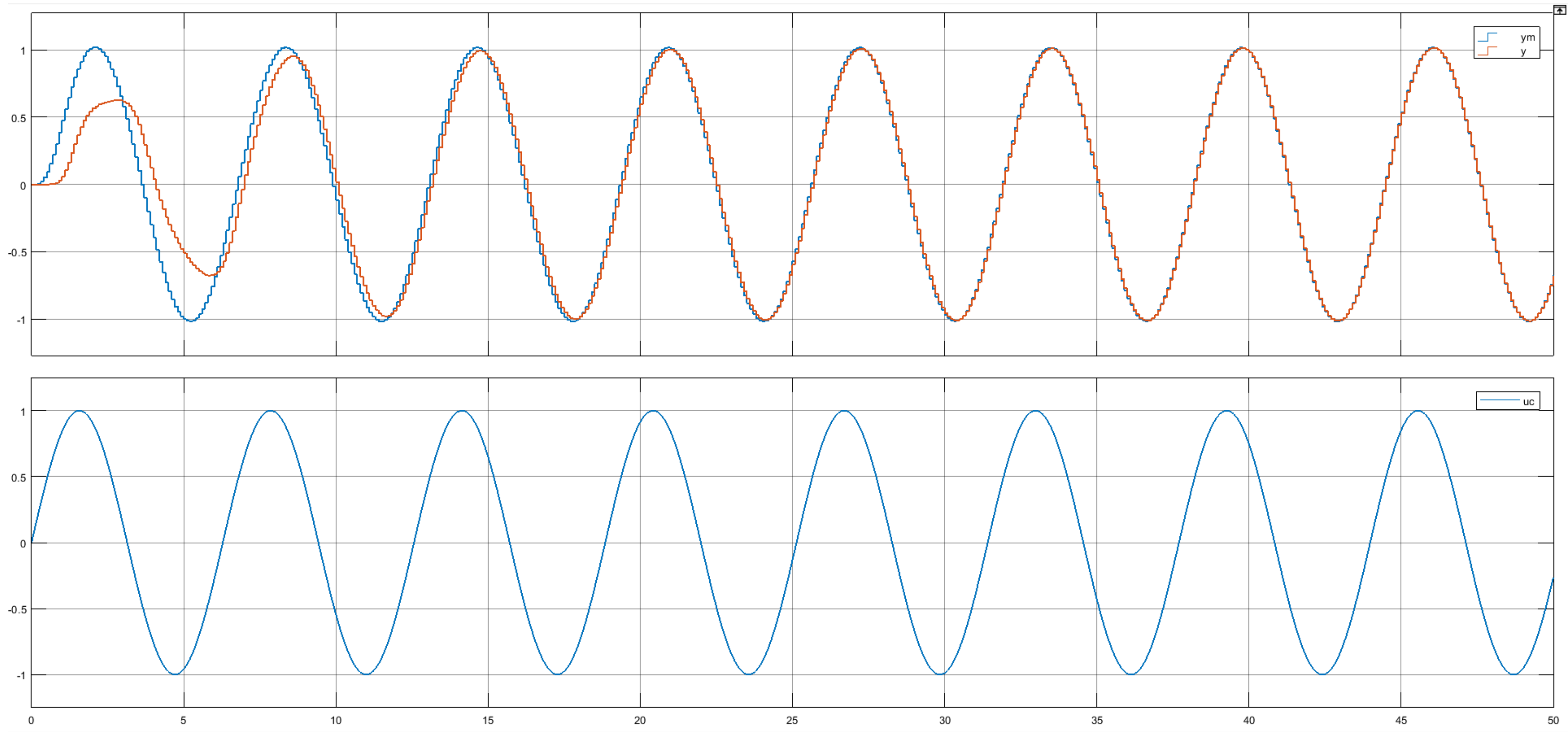
wn=3.2	
si=0.6266	



wn=3.2; si=0.6266;



$T_s = 10\text{ms}$



$T_s = 10\text{ms}$

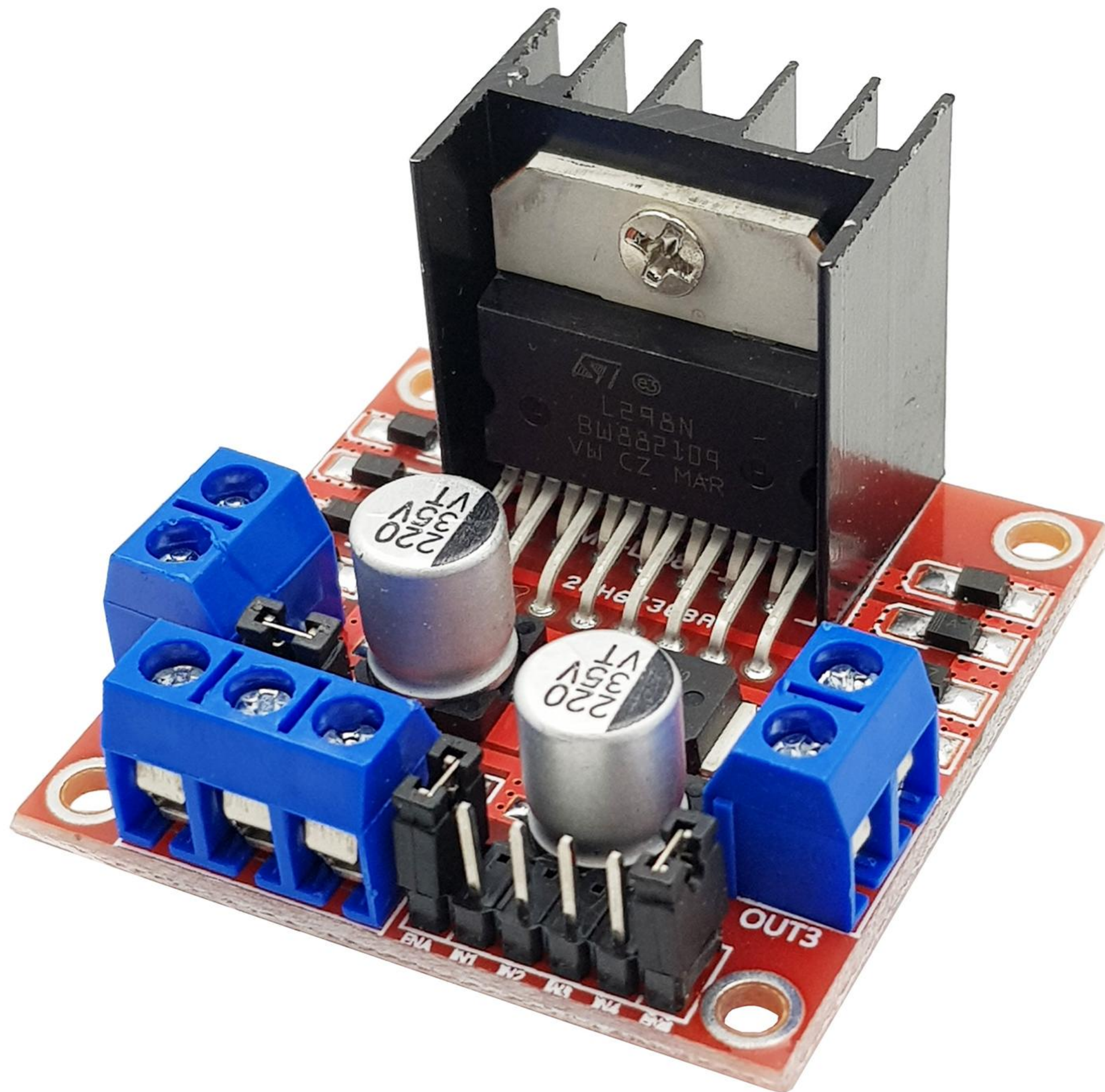
Design and Microcontroller Programming

01

HARDWARE

02

SOFTWARE



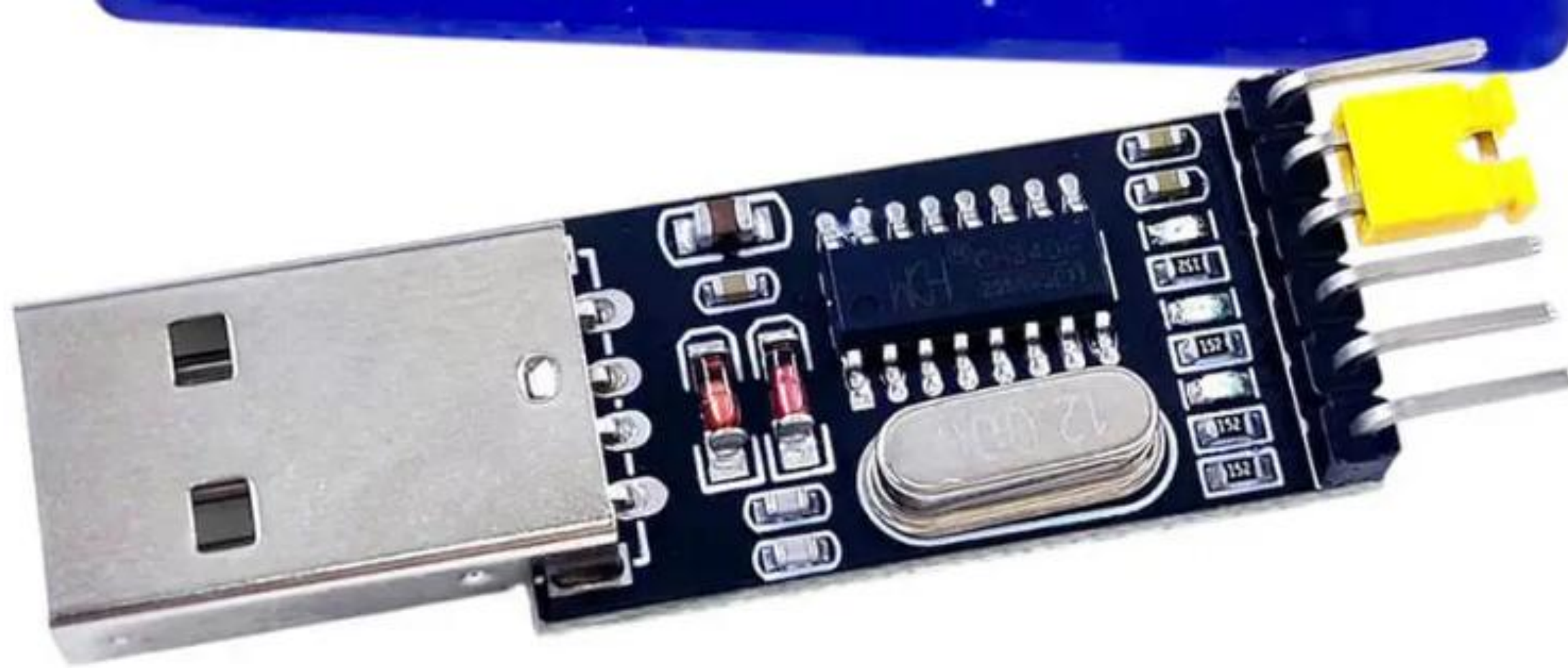
**L298N MOTOR
DRIVER
CIRCUIT**



- GA25-370 MOTOR
280RPM (WITH
GEAR
REDUCTION) –
21.3:1
- TWO-CHANNEL
ENCODER
(CHANNELS A
AND B), PPR = 11



- Model: OEM ADS0248-W120200A
- Input Voltage: 100–240 VAC, 50/60 Hz
- Output Voltage: 12 VDC (recommend ~70% load for continuous use)
- Max Output Current: 2 A
- Max Power: 24 W (continuous load ~70% for optimal durability)
- Power Type: Switching Power Supply
- Output Connector: Standard 5.5 x 2.1–2.5 mm DC Jack
- Cable Length: 1.5 m
- Dimensions: 72 x 34 x 50 mm



- Uses 5V DC from USB port (max 500mA).
- Signal Pins:
- 5V: Power from USB (max 500mA).
- VCC: TTL logic level selector (connect to 5V for 5V TTL, or 3.3V for 3.3V TTL).
- 3V3: 3.3V output (low current, max 100mA), mainly for logic level setting, not power supply.
- TXD: UART transmit (connects to RX of TTL devices).
- RXD: UART receive (connects to TX of TTL devices).
- GND: Ground (0V).
- Integrated TX and RX indicator LEDs.
- Compatible with Windows, Mac, and Linux.



STM32F103C8T6 - BLUE PILL

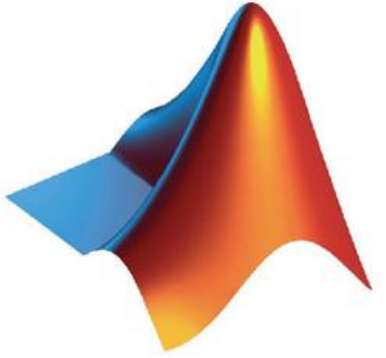
μVision® 5

Integrated
Development
Environment

Copyright © 2010-2016 ARM Ltd. All rights reserved.
This product is protected by US and international laws.

ARMKEIL
Microcontroller Tools

 C#
Microsoft
Visual Studio

MATLAB® 
& SIMULINK®



CODING

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) //Ngat ngoai dem xung
{
    if (GPIO_Pin == GPIO_PIN_10) // PA10 = EXTI10
    {
        flag_pulse = 1; // Set co bao co xung
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) //Ngat 100ms tinh toc do
{
    if (htim->Instance == TIM1) //Ngat 100ms tinh toc do
    {
        flag_timer_100ms = 1;
        //HAL_GPIO_TogglePin(GPIOC,GPIO_PIN_13);
        // Danh dau da den chu ky 100ms
    }
    if (htim->Instance == TIM3) //Ngat 0.2s truyen uart
    {
        uart_flag_tx=1;
    }
}

float calculate_motor_speed_rpm() {
    const float pulses_per_encoder_rev = 11.0f;
    const float gear_ratio = 21.3f;
    const float sampling_time_sec = 0.1; // 100ms

    float encoder_rps = encoder_count / pulses_per_encoder_rev / sampling_time_sec;

    // Toc do truc dau ra
    float motor_rpm = encoder_rps * 60.0f / gear_ratio;

    // Reset encoder count sau m?i chu k?
    encoder_count = 0;

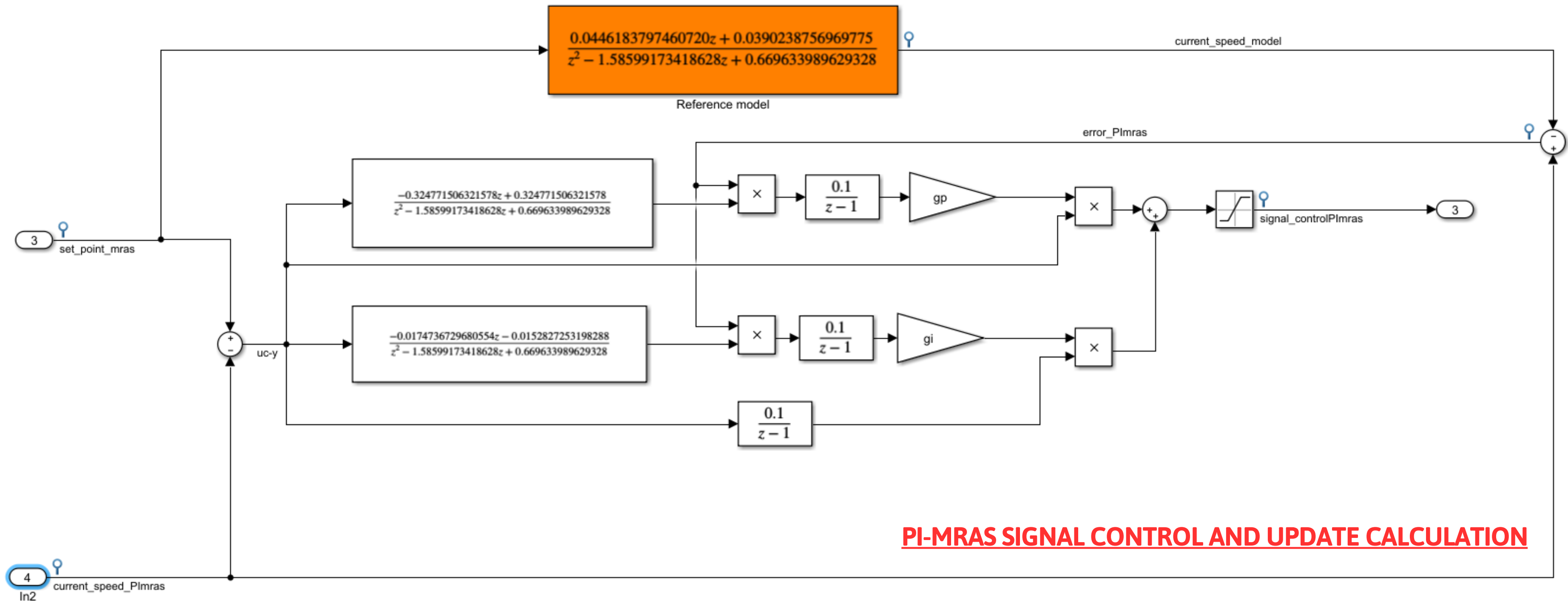
    return motor_rpm;
}
```

Encoder Pulse Counting

Speed Calculation and Data Transmission

Function for Speed Calculation

CODING (BASE SIMULIK)



PI-MRAS SIGNAL CONTROL AND UPDATE CALCULATION

```
/* Exported block parameters */
```

```
real_T gi = 0.0;
```

```
/* Variable: gi
```

```
 * Referenced by: '<Root>/gamma3'
```

```
 */
```

```
real_T gp = 0.0;
```

```
/* Variable: gp
```

```
 * Referenced by: '<Root>/gamma2'
```

```
 */
```

```
real_T ki = 0.0;
```

```
/* Variable: ki
```

```
 * Referenced by: '<S1>/Integral Gain'
```

```
 */
```

```
real_T kp = 0.0;
```

```
/* Variable: kp
```

```
 * Referenced by: '<S1>/Proportional Gain'
```

```
 */
```

```
rtb_DiscreteTransferFcn3 = 0.1 * rtDW.DiscreteTransferFcn7_states * gp * ucy +  
    0.1 * rtDW.DiscreteTransferFcn6_states * gi * (0.1 *  
    rtDW.DiscreteTransferFcn5_states);
```

```
/* Saturate: '<Root>/Saturation2' */
```

```
if (rtb_DiscreteTransferFcn3 > 100.0) {
```

```
    signal_controlPImras = 100.0;
```

```
} else if (rtb_DiscreteTransferFcn3 < 0.0) {
```

```
    signal_controlPImras = 0.0;
```

```
} else {
```

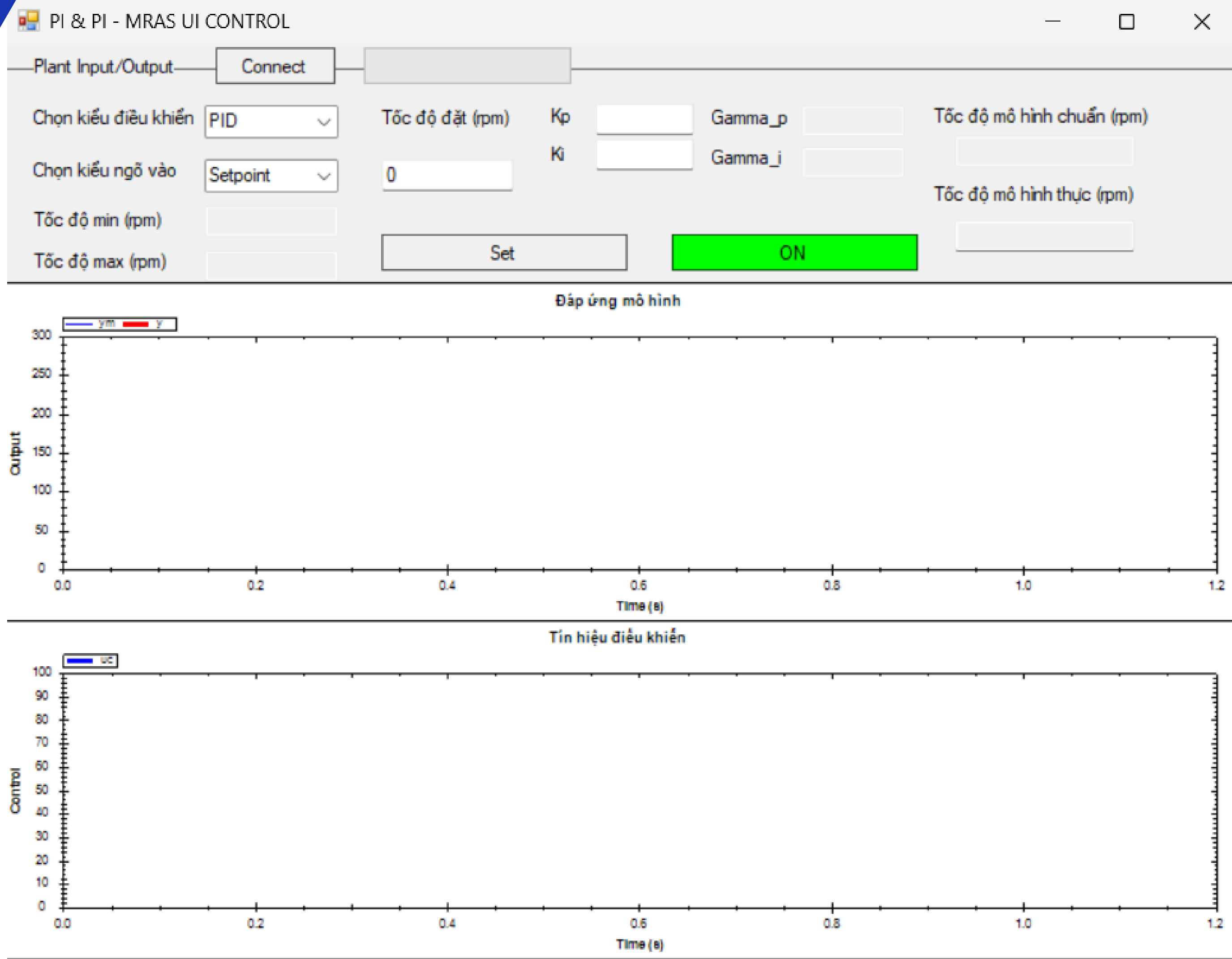
```
    signal_controlPImras = rtb_DiscreteTransferFcn3;
```

```
}
```

GLOBAL PARAMETER

PI - MRAS SIGNAL CALCULATION

UI INTERFACE



param 1	0/1	Động cơ on hoặc off	
param 2	0/1	Kiểu điều khiển: PI thường 0, PI mras 1	
param 3	0/1/2	Kiểu input: const 0, sine 1, pulse 2	
param 4	thuộc R	kp	
param 5	thuộc R	ki	
param 6	thuộc R	gp	
param 7	thuộc R	gi	
param 8	thuộc R	amp_max (pulse) 50-->250 > amp_min	
param 9	thuộc R	amp_min (pulse) 50-->250 < amp_max	
param 10	thuộc R	set_point (const) 50-->250	Chuỗi kết thúc bằng R
Ví dụ: 1,0,0,0.5,0.3,0.2,0.9,250,50,100R			
Động cơ ON, điều khiển PI thường, kiểu input const,kp,ki,gp,gi tương ứng,amp_max 250, amp_min 50, set_point = 100R			

```

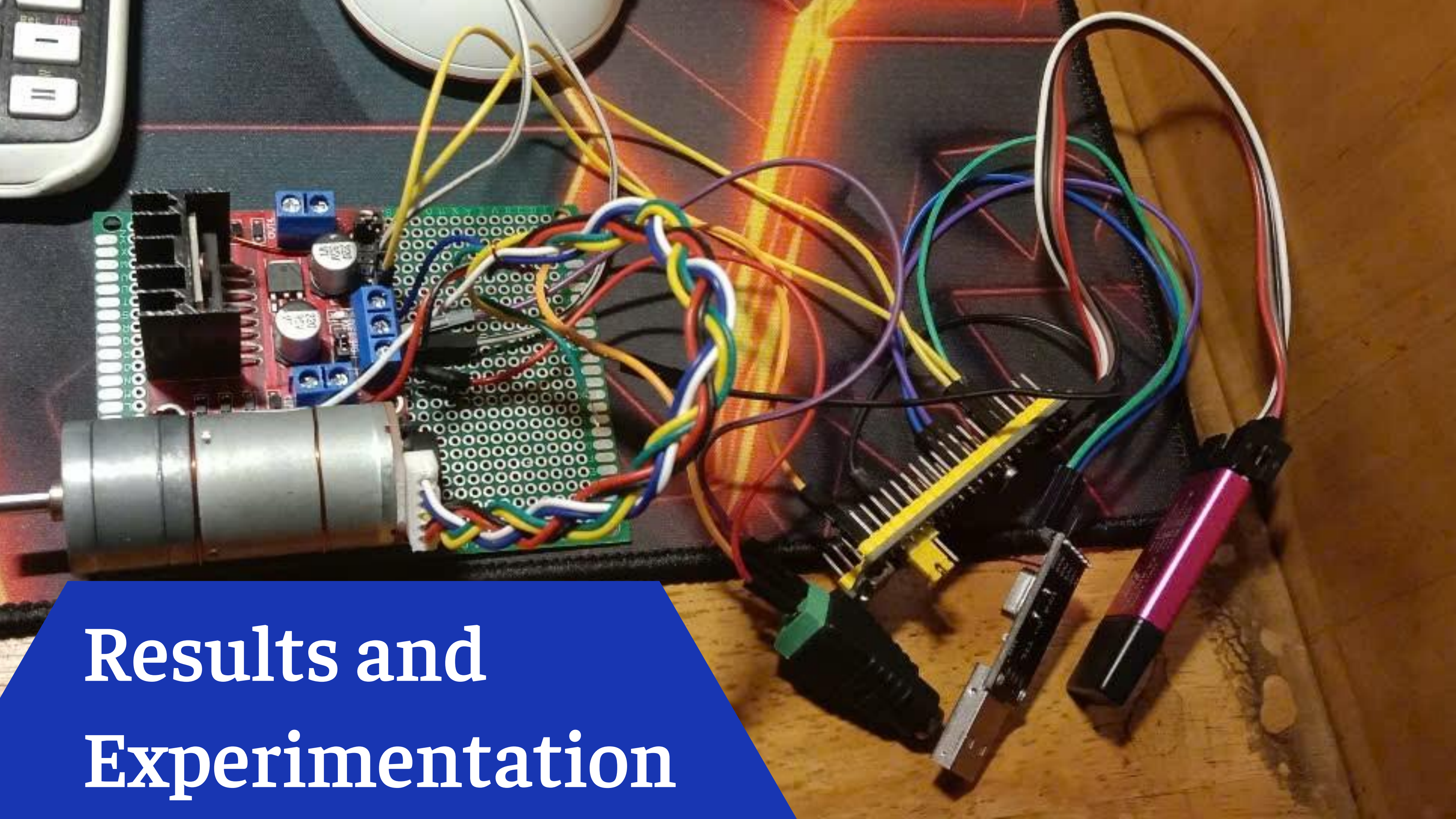
void Parse_UART_Command(char* str) {
    char* token;
    uint8_t field = 0;

    token = strtok(str, ",");
    while (token != NULL) {
        switch (field) {
            case 0: motor_on = atoi(token); break;
            case 1: controller_type = atoi(token); break;
            case 2: input_type = atoi(token); break;
            case 3: kp = atof(token); break;
            case 4: ki = atof(token); break;
            case 5: gp = atof(token); break;
            case 6: gi = atof(token); break;
            case 7: amp_max = atof(token); break;
            case 8: amp_min = atof(token); break;
            case 9: set_point = atof(token); break;
            default: break;
        }

        token = strtok(NULL, ",");
        field++;
    }
}

```

UART_received_processing_data



Results and Experimentation

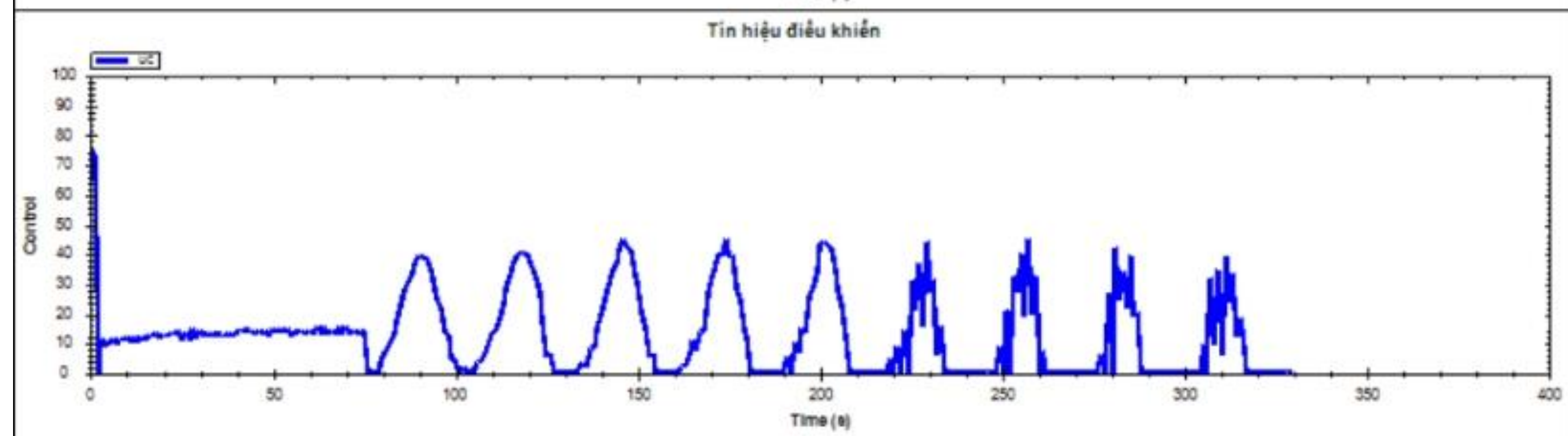
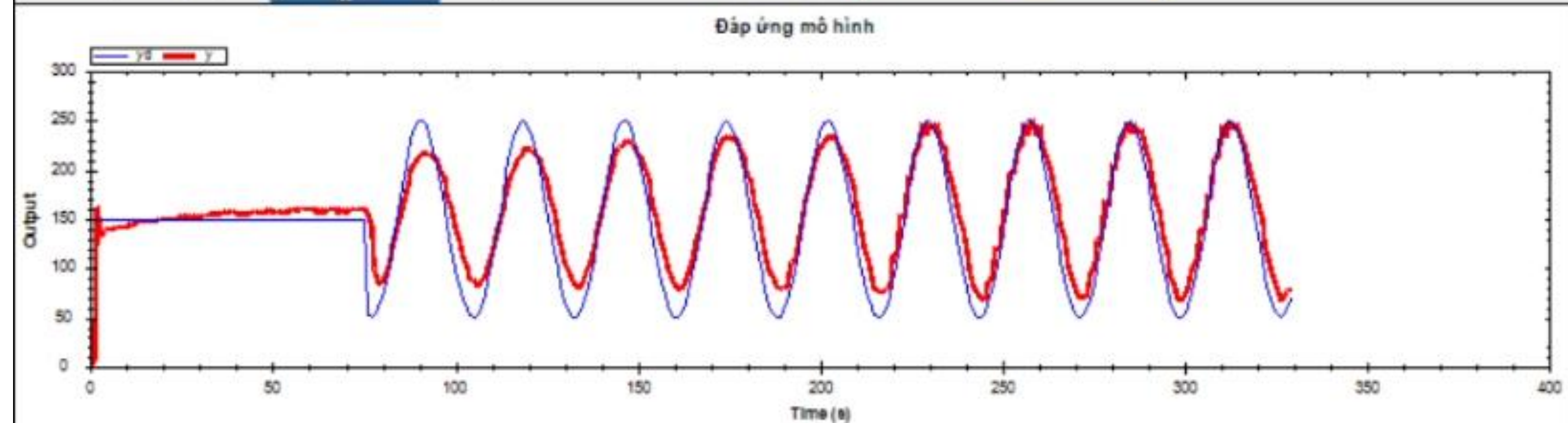
Plant Input/Output

Chọn kiểu điều khiển Tốc độ đặt (rpm) K_p Γ_p Tốc độ mô hình chuẩn (rpm)

Chọn kiểu ngõ vào K_i Γ_i Tốc độ mô hình thực (rpm)

Tốc độ min (rpm) ON

Tốc độ max (rpm)



Plant Input/Output

Connect

Chọn kiểu điều khiển

PI-MRAS

Tốc độ đặt (rpm)

Kp

1

Gamma_p

0.001

Tốc độ mô hình chuẩn (rpm)

Chọn kiểu ngõ vào

Pulse

100

Ki

0.001

Gamma_i

0.002

150

Tốc độ min (rpm)

75

Tốc độ max (rpm)

150

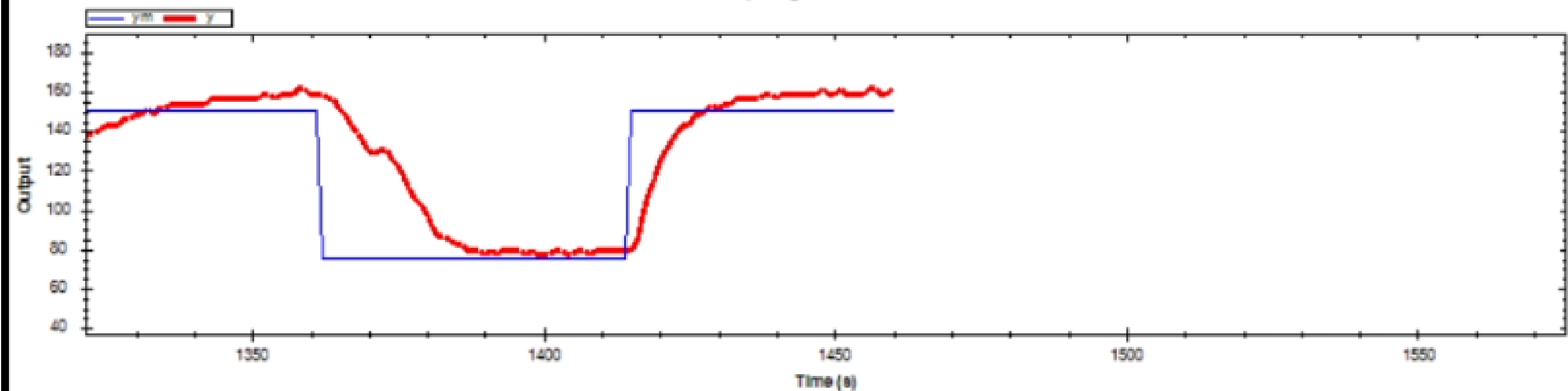
Set

ON

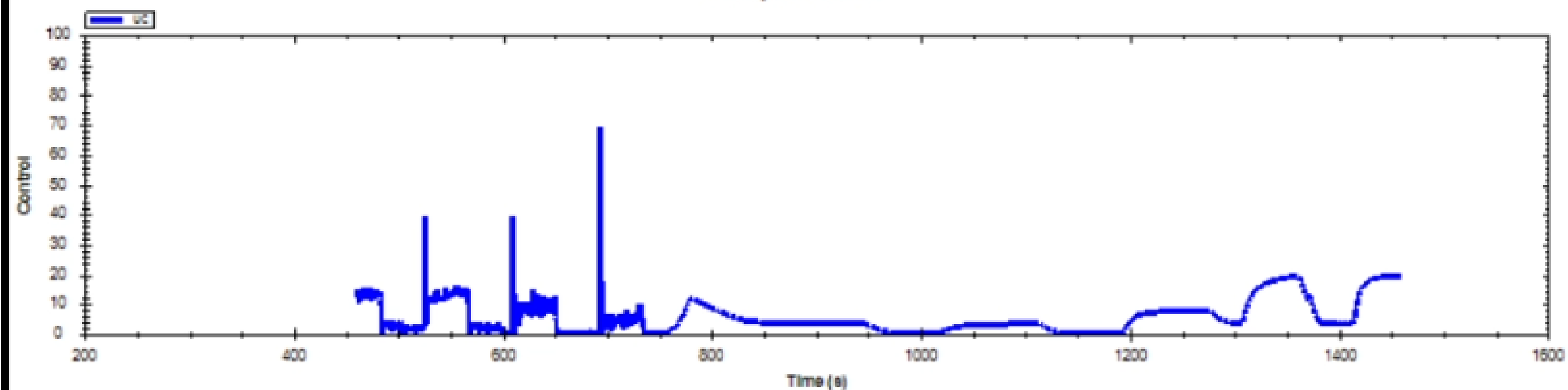
Tốc độ mô hình thực (rpm)

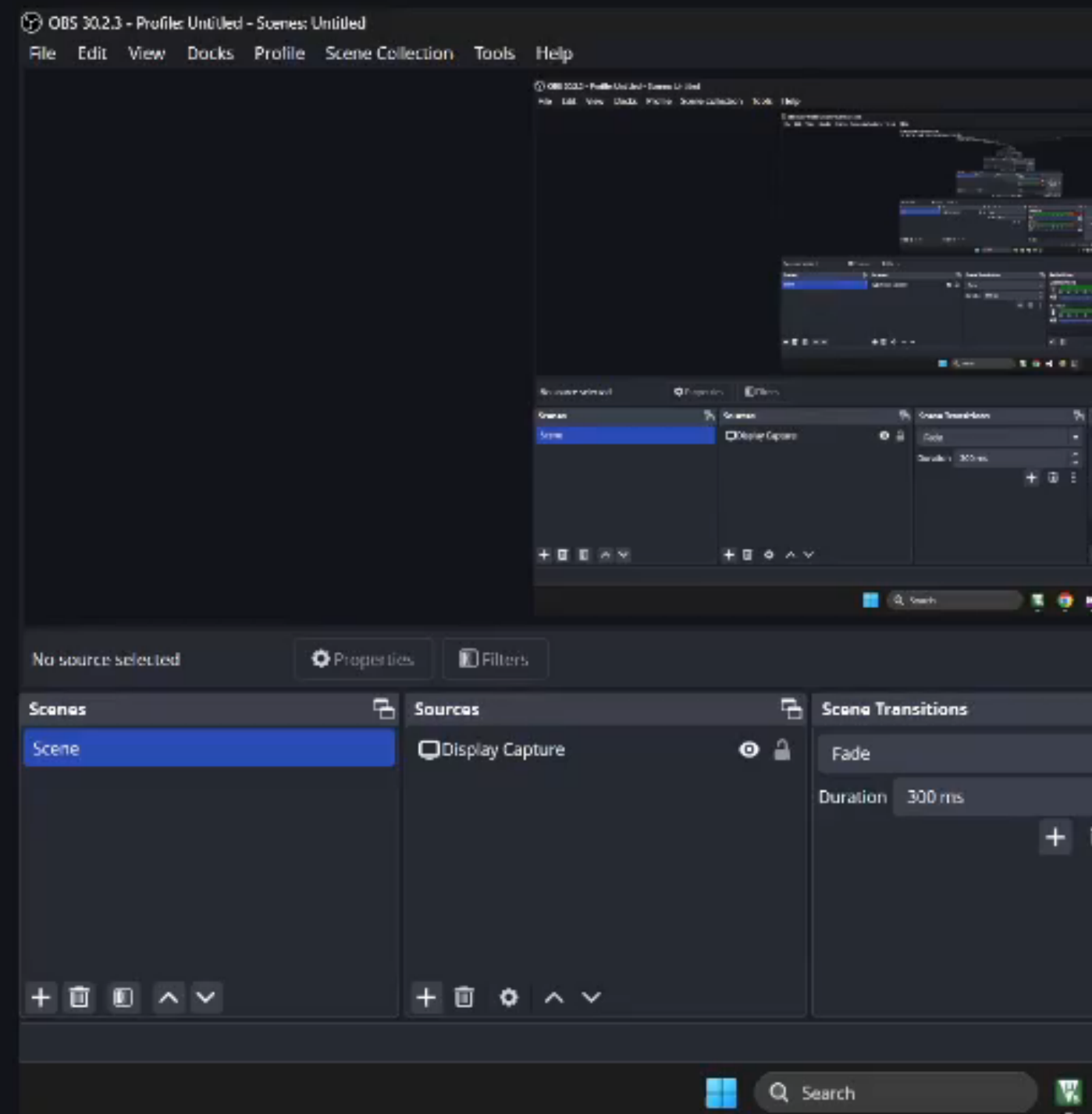
161.33

Đáp ứng mô hình



Tín hiệu điều khiển





No source selected

 Properties

 Filters

Scenes



Scene

Sources



Display Capture



Scene Transitions

Fade

Duration 300 ms

THANK YOU!



HCMUT - UNIVERSITY OF TECHNOLOGY

